

Runbook — Demo "Rome Future Week" (Crm-A Console)

Eseguibile dalla suite integrata. Il provider telefonico fa l'AI della conversazione; la Crm-A Console è il cervello CRM (lookup, registrazione, orchestrazione) ed espone il webhook.

0. Prerequisiti

```
# 1) Segreto webhook (necessario per auth ingress + demo seed + campagne)
export CRM_A_PHONE_WEBHOOK_SECRET=<secret-demo>

# 2) Dial telefonico outbound (solo telefono; il provider la genera)
export CRM_A_PHONE_OUTBOUND_URL=https://<provider-outbound>
export CRM_A_PHONE_OUTBOUND_SECRET=<provider-secret>

# 3) Telegram outbound: canale telegram + bot connessi nel gateway openclaw
# (openclaw.json → channels.telegram.token). NON passa dal provider.
```

Avvio console e **seed** dello scenario:

```
curl -X POST localhost:3100/api/demo/seed \
  -H "Authorization: Bearer $CRM_A_PHONE_WEBHOOK_SECRET"
# → products, people, order, segment
```

Atto 1 — Primo contatto (chiamata in ingresso + consenso)

La parte telefonica chiama questi webhook; mostra lo scambio.

```
# 1a) in ingresso: numero sconosciuto → la Console crea la Persona +
contesto
curl -sX POST localhost:3100/api/webhooks/phone \
```

```
-H "Authorization: Bearer $CRM_A_PHONE_WEBHOOK_SECRET" -H 'content-type: application/json' \
-d '{"action":"inbound","callId":"vc-1","from":{"phone":"+39311222333"}}'
# → { person: { id, matched:"created" }, context:"Nuovo contatto registrato...", callStatus:"continue" }

# 1b) fine chiamata: consenso + preferenza + dati → registra Call + aggiorna anagrafica
curl -sX POST localhost:3100/api/webhooks/phone \
-H "Authorization: Bearer $CRM_A_PHONE_WEBHOOK_SECRET" -H 'content-type: application/json' \
-d '{"action":"completed","callId":"vc-1","from":{"phone":"+39311222333"},"durationSec":150,
  "data":
{"name":"Lorenzo","email":"lorenzo@example.com","preferredContact":"telegram",
  "marketingOptIn":true,"summary":"Vuole essere avvisato al lancio Galaxy"}}'
# → { ok, interactionId, personId, actions: ["interaction_recorded","person_updated"] }
```

Verifica in console: profilo "Lorenzo" (Preferenza=telegram, Opt-in=true) + interazione **Call** nella timeline.

Atto 2 — Il CRM diventa memoria (Copilot)

Aprire il Copilot (chat) e chiedere, come nel demo:

"Stiamo lanciando il nuovo Samsung Galaxy. Qual è il pubblico migliore a cui proporlo?"

Il Copilot usa la skill **crm** sui dati seedati: segmento "Lancio Samsung Galaxy" (opt-in=true), acquirenti S26/S25 (ordini), preferenze di canale. Risponde con la lista; Lorenzo è incluso.

Atto 3 — Orchestrazione (campagna multicanale)

Passo 0 — Export del brief di marketing (contenuto per l'ambiente telefonico): La conoscenza del prodotto viaggia come MD importato nell'ambiente telefonico (che fa

l'AI della conversazione), non come lookup a runtime.

```
curl -s -H "Authorization: Bearer $CRM_A_PHONE_WEBHOOK_SECRET" \
  "localhost:3100/api/marketing/brief?promotedSku=SAM-S27&previousSku=SAM-S26" \
  -o brief-galaxy-s27.md
# Contiene: prodotto+lancio, copy ufficiale
(caratteristiche/differenze/vantaggi/limiti,
# promo, link acquisto), confronto vs S26, pubblico, esempio memoria Atto 5.
# → importare brief-galaxy-s27.md nell'ambiente telefonico (briefing
dell'AI).
```

Passo 1 — Invio multicanale per preferenza di canale:

```
curl -sX POST localhost:3100/api/campaigns/send-multichannel \
  -H "Authorization: Bearer $CRM_A_PHONE_WEBHOOK_SECRET" -H 'content-type:
application/json' \
  -d '{"segmentEntryId":"<id segmento>","subject":"Samsung Galaxy 27",
      "body":"Disponibile dal 18 ottobre. Promo per i primi clienti."}'
# → { sent, telegram: <a chi ha scelto telegram>, email: <a chi ha scelto
email>, failed:[] }
```

- Telegram → runtime openclaw (sessione `phone:<e164>`, invio reale richiede bot connesso).
- email → SES (campagna esistente).

Mostra la segmentazione per canale: chi ha scelto Telegram riceve su Telegram, chi email su email.

Atto 4 — Il cliente riceve il messaggio e conversa

L'ambiente telefonico risponde alla domanda del cliente usando il **brief MD importato all'Atto 3** (caratteristiche, differenze vs S26, vantaggi, limiti). La Console, su messaggio in ingresso, restituisce il contesto del cliente:

```
curl -sX POST localhost:3100/api/webhooks/phone \
  -H "Authorization: Bearer $CRM_A_PHONE_WEBHOOK_SECRET" -H 'content-type:
application/json' \
  -d '{"action":"message","messageId":"tg-1","contact":
{"phone":"+393312345678","name":"Lorenzo"},

```

```
"text": "Grazie, l'offerta mi interessa. Vale la pena?"]}
```

```
# → { person: profilo cliente, context: contesto CRM
```

```
(identità/storico/acquisti), replyFor: "message" }
```

```
# Il contenuto del prodotto viene dal brief MD importato (Atto 3).
```

Acquisto → si registra (webhook `completed` con `data.order`, oppure evento `Purchase` via `/api/crm/events`).

Atto 5 — Dopo l'acquisto (riconoscimento memoria)

La parte telefonica richiama `inbound` per Lorenzo (numero noto):

```
curl -sX POST localhost:3100/api/webhooks/phone \
  -H "Authorization: Bearer $CRM_A_PHONE_WEBHOOK_SECRET" -H 'content-type:
application/json' \
  -d '{"action": "inbound", "callId": "vc-5", "from":
{"phone": "+393312345678", "name": "Lorenzo"}}'
# → { person: { name: "Lorenzo", lastOrder: { productName: "Samsung Galaxy
S26",
#       orderedAt, status: "Shipped", courier: "GLS",
#       deliveryStatus: "Corriere in carico oggi; consegna prevista domani
entro le 18" } },
#       context: "Cliente esistente: Lorenzo... ultimo acquisto: Samsung Galaxy
S26... corriere GLS..." }
```

L'Al telefonica pronuncia "Bentornato Lorenzo... il corriere... consegna entro le 18".

Config rapida

Var	Ruolo
<code>CRM_A_PHONE_WEBHOOK_SECRET</code>	auth ingress + seed + campagne
<code>CRM_A_PHONE_OUTBOUND_URL / _SECRET</code>	dial telefonico (provider)
Telegram bot nel gateway	outbound Telegram (runtime)

Note NLPearl (verificate in live)

- **Phone Number ID:** il campo `direction` non è inbound/outbound. Non tutti i numeri dell'account sono autorizzati per l'outbound (`400 "not authorized for outbound calls"`). Verificato funzionante per entrambe le direzioni: `686fd112a91849a9e59a5353` (+39654547159). Scegli il `phoneNumberId` dalle voci prive di errore 400 alla creazione Pearl.
- **Variabili:** `firstName/email` sono built-in delle lead (non dichiarabili); `variables` deve essere un array non vuoto (usa una custom operational, es. `customerNote`, group 2).
- **Creazione Pearl:** richiede `pearl.timeZone` (Windows Time), `pearl.companyDescription` e (inbound) `inbound.waitingSentence`. La risposta di `POST /Pearl/Voice` è il Pearl ID come plain text (non JSON).
- **Prerequisito E2E:** le URL webhook necessitano di un'origine **pubblica** raggiungibile da NLPearl (`CRM_A_CONSOLE_PUBLIC_URL` + `funnel/tunnel/deploy`). Finché non c'è, le Pearl si creano (paused, sintassi ok) ma i callback restano non collaudabili.
- **Test di creazione live** (paused, senza chiamate): inbound `6a79f52d13744b2317945734` (type 1), outbound `6a79f5dd13744b2317945739` (type 2) — rimossi/o da dashboard a fine demo.